FOUNDATION OF JAVA LAB SHEET – 8
Generic

Q1. Generic Box Create a generic class Box that stores one value. Create objects for Integer, String and Double and display all three values.

```
class Box<T> {
    T value;

    Box(T value) {
        this.value = value;
    }

    void display() {
        System.out.println(value);
    }
}

public class Main {
    public static void main(String[] args) {
        Box<Integer> b1 = new Box<>(100);
        Box<String> b2 = new Box<>("Java");
        Box<Double> b3 = new Box<>(25.5);

        b1.display();
        b2.display();
        b3.display();
    }
}
```

OUTPUT:
100
Java
25.5

Q2. Generic Result Create a generic class Result that stores marks. Create Result and Result objects and display the marks.

```
class Result<T> {
```

```
    T marks;

    Result(T marks) {
        this.marks = marks;
    }

    void display() {
        System.out.println("Marks: " + marks);
    }
}

public class Main {
    public static void main(String[] args) {
        Result<Integer> r1 = new Result<>(85);
        Result<Double> r2 = new Result<>(92.5);

        r1.display();
        r2.display();
    }
}
```

OUTPUT:
Marks: 85
Marks: 92.5

Q3. Generic Student Create a generic class Student where T represents the student ID. Create a Student object and display the ID

```
class Student<T> {
    T id;

    Student(T id) {
        this.id = id;
    }

    void display() {
        System.out.println("Student ID: " + id);
    }
}

public class Main {
```

```
    public static void main(String[] args) {
        Student<Integer> s = new Student<>(101);
        s.display();
    }
}
```

OUTPUT:
Student ID: 101

```
class Pair<T, U> {
    T id;
    U name;

    Pair(T id, U name) {
        this.id = id;
        this.name = name;
    }

    void display() {
        System.out.println("Employee ID: " + id);
        System.out.println("Employee Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Pair<Integer, String> p = new Pair<>(101, "Sumit");
        p.display();
    }
}
```

OUTPUT:
Employee ID: 101
Employee Name: Sumit

Q5. Product Pair Create a generic class with two type parameters to store Product ID and Product Price. Create at least three product objects.

```
class Product<T, U> {
```

```
    T id;
    U price;

    Product(T id, U price) {
        this.id = id;
        this.price = price;
    }

    void display() {
        System.out.println("Product ID: " + id + ", Price: " + price);
    }
}

public class Main {
    public static void main(String[] args) {
        Product<Integer, Double> p1 = new Product<>(101, 499.99);
        Product<Integer, Double> p2 = new Product<>(102, 799.50);
        Product<Integer, Double> p3 = new Product<>(103, 999.00);

        p1.display();
        p2.display();
        p3.display();
    }
}
```

OUTPUT:
Product ID: 101, Price: 499.99
Product ID: 102, Price: 799.5
Product ID: 103, Price: 999.0

Q6. Generic Employee Create Employee where T represents Employee ID and U represents Employee Name. Add a display() method.

```
class Employee<T, U> {
    T id;
    U name;

    Employee(T id, U name) {
        this.id = id;
        this.name = name;
    }
```

```
    void display() {
        System.out.println("Employee ID: " + id);
        System.out.println("Employee Name: " + name);
    }
}

public class Main {
    public static void main(String[] args) {
        Employee<Integer, String> e = new Employee<>(101, "Sumit");
        e.display();
    }
}
```

OUTPUT:
Employee ID: 101
Employee Name: Sumit

Q7. Generic Method Create a generic method display() that can display Integer, Double, String and Character values.

```
public class Main {

    static <T> void display(T value) {
        System.out.println(value);
    }

    public static void main(String[] args) {
        display(100);
        display(25.5);
        display("Java");
        display('A');
    }
}
```

OUTPUT:
100
25.5
Java
A

Q8. Generic Method for Three Values Create a generic method that accepts three values of the same type and displays them. Test it using Integers, Strings and Doubles.

```
public class Main {

    static <T> void display(T a, T b, T c) {
        System.out.println(a + " " + b + " " + c);
    }

    public static void main(String[] args) {
        display(10, 20, 30);
        display("Java", "Python", "C++");
        display(10.5, 20.5, 30.5);
    }
}
```

OUTPUT:
10 20 30
Java Python C++
10.5 20.5 30.5

Q9. Generic ArrayList Create an ArrayList. Add 10 numbers and display them using a loop.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();

        for (int i = 1; i <= 10; i++)
            numbers.add(i);

        for (Integer n : numbers)
            System.out.println(n);
    }
}
```

OUTPUT:
1
2

3
4
5
6
7
8
9
10

Q10. Generic String List Create an ArrayList containing five student names. Perform add(), get(), set(), remove() and size(), and display the results.

```
import java.util.ArrayList;

public class Main {
    public static void main(String[] args) {
        ArrayList<String> students = new ArrayList<>();

        // add()
        students.add("Sumit");
        students.add("Rahul");
        students.add("Aman");
        students.add("Rohit");
        students.add("Karan");

        // get()
        System.out.println("Get: " + students.get(0));

        // set()
        students.set(1, "Arjun");

        // remove()
        students.remove(2);

        // size()
        System.out.println("Size: " + students.size());
        System.out.println("Students: " + students);
    }
}
```

OUTPUT:

Get: Sumit
Size: 4
Students: [Sumit, Arjun, Rohit, Karan]

Q11. Search in Generic List Create ArrayList containing ten numbers. Ask the user for a number and use contains() to determine whether the number exists.

```
import java.util.ArrayList;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        Scanner sc = new Scanner(System.in);

        for (int i = 1; i <= 10; i++)
            numbers.add(i);

        System.out.print("Enter number: ");
        int n = sc.nextInt();

        if (numbers.contains(n))
            System.out.println("Number exists");
        else
            System.out.println("Number not found");
    }
}
```

OUTPUT:
Enter number: 7
Number exists

Q12. Generic Container Create Container with add() and get() methods. Test it using Integer, String and Double.

```
class Container<T> {
    T value;

    void add(T value) {
        this.value = value;
    }
```

```
    T get() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) {
        Container<Integer> c1 = new Container<>();
        Container<String> c2 = new Container<>();
        Container<Double> c3 = new Container<>();

        c1.add(100);
        c2.add("Java");
        c3.add(25.5);

        System.out.println(c1.get());
        System.out.println(c2.get());
        System.out.println(c3.get());
    }
}
```

OUTPUT:
100
Java
25.5

Q13. Generic Class with Three Parameters Create Student where T = ID, U = Name and V = Marks. Create at least three student objects with suitable types.

```
class Student<T, U, V> {
    T id;
    U name;
    V marks;

    Student(T id, U name, V marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }
```

```
    void display() {
        System.out.println(id + " " + name + " " + marks);
    }
}

public class Main {
    public static void main(String[] args) {
        Student<Integer, String, Double> s1 =
            new Student<>(101, "Sumit", 85.5);

        Student<Integer, String, Double> s2 =
            new Student<>(102, "Rahul", 78.0);

        Student<Integer, String, Double> s3 =
            new Student<>(103, "Aman", 92.5);

        s1.display();
        s2.display();
        s3.display();
    }
}
```

OUTPUT:
101 Sumit 85.5
102 Rahul 78.0
103 Aman 92.5

Q14. Generic Method and ArrayList Create an ArrayList and a generic method that displays all elements of the list. Test it with ArrayList and ArrayList.

```
import java.util.ArrayList;

public class Main {

    static <T> void display(ArrayList<T> list) {
        for (T item : list)
            System.out.println(item);
    }

    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
```

```
        numbers.add(10);
        numbers.add(20);
        numbers.add(30);

        ArrayList<String> names = new ArrayList<>();
        names.add("Sumit");
        names.add("Rahul");
        names.add("Aman");

        display(numbers);
        display(names);
    }
}
```

OUTPUT:
10
20
30
Sumit
Rahul
Aman

Q15. Mini Generic Student Management Program Create a small student management program using Student, where T = Student ID, U = Student Name and V = Marks. Create at least five students, store them using an appropriate collection, display all students, search for a student, display the total number of students, and demonstrate type safety. Use Generic Class, Multiple Type Parameters, ArrayList, add(), get(), size(), contains(), loops, methods, constructors and type safety.

```
import java.util.ArrayList;

class Student<T, U, V> {
    T id;
    U name;
    V marks;

    Student(T id, U name, V marks) {
        this.id = id;
        this.name = name;
        this.marks = marks;
    }
```

```
    void display() {
        System.out.println(id + " " + name + " " + marks);
    }
}

public class Main {

    static void displayAll(ArrayList<Student<Integer, String, Double>> list) {
        for (Student<Integer, String, Double> s : list)
            s.display();
    }

    static void search(ArrayList<Student<Integer, String, Double>> list, int id) {
        for (Student<Integer, String, Double> s : list) {
            if (s.id.equals(id)) {
                System.out.println("Student Found: " + s.name);
                return;
            }
        }
        System.out.println("Student Not Found");
    }

    public static void main(String[] args) {

        ArrayList<Student<Integer, String, Double>> students =
                new ArrayList<>();

        students.add(new Student<>(101, "Sumit", 85.5));
        students.add(new Student<>(102, "Rahul", 78.0));
        students.add(new Student<>(103, "Aman", 90.5));
        students.add(new Student<>(104, "Rohit", 75.0));
        students.add(new Student<>(105, "Karan", 88.5));

        System.out.println("All Students:");
        displayAll(students);

        System.out.println("\nSearch:");
        search(students, 103);

        System.out.println("\nTotal Students: " + students.size());

        System.out.println("Contains Student ID 101: " +
                students.get(0).id.equals(101));
```

```
        // Type safety:
        // students.add(new Student<>("106", "Neha", "95")); // Error
    }
}
```

OUTPUT:
All Students:
101 Sumit 85.5
102 Rahul 78.0
103 Aman 90.5
104 Rohit 75.0
105 Karan 88.5

Search:
Student Found: Aman

Total Students: 5
Contains Student ID 101: true